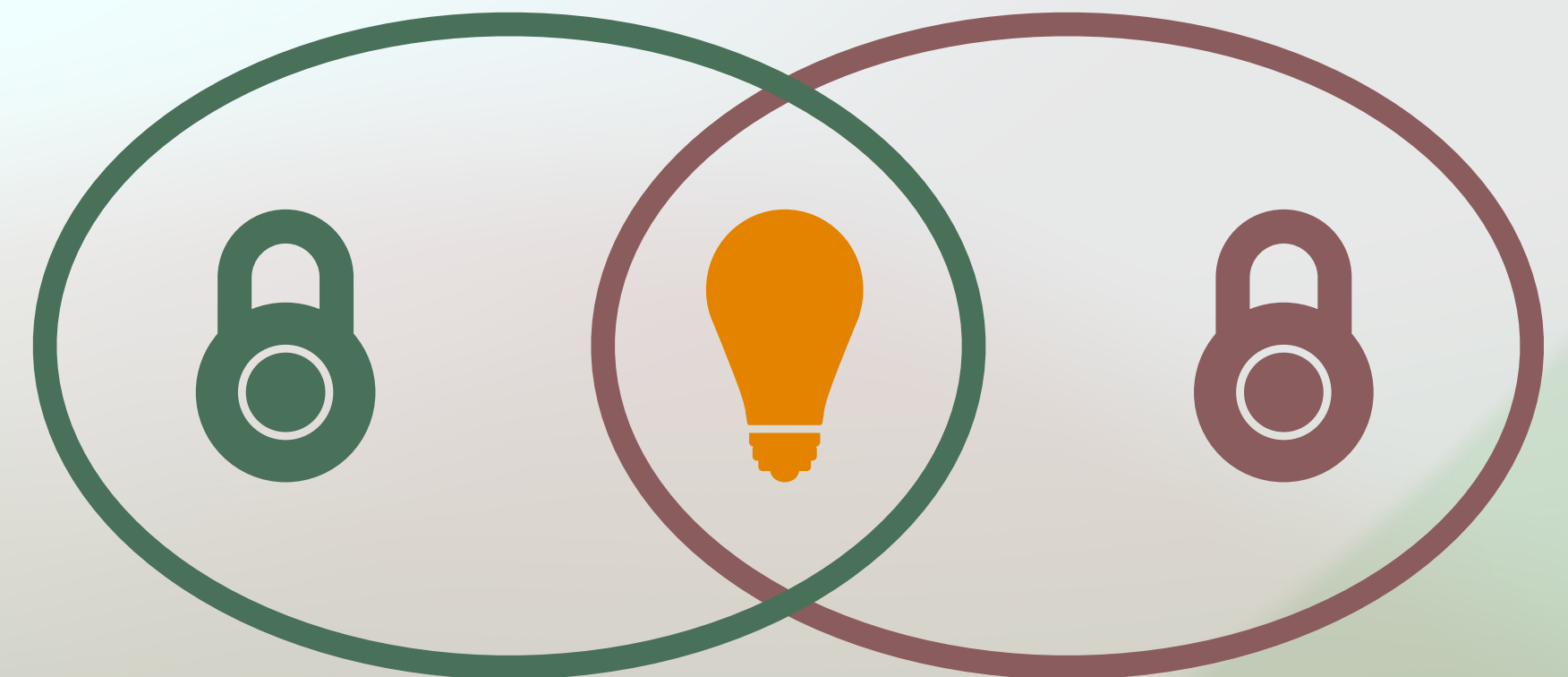


Just-In-Time-OPRFs and a Modular Framework for Fast Private Set Intersection

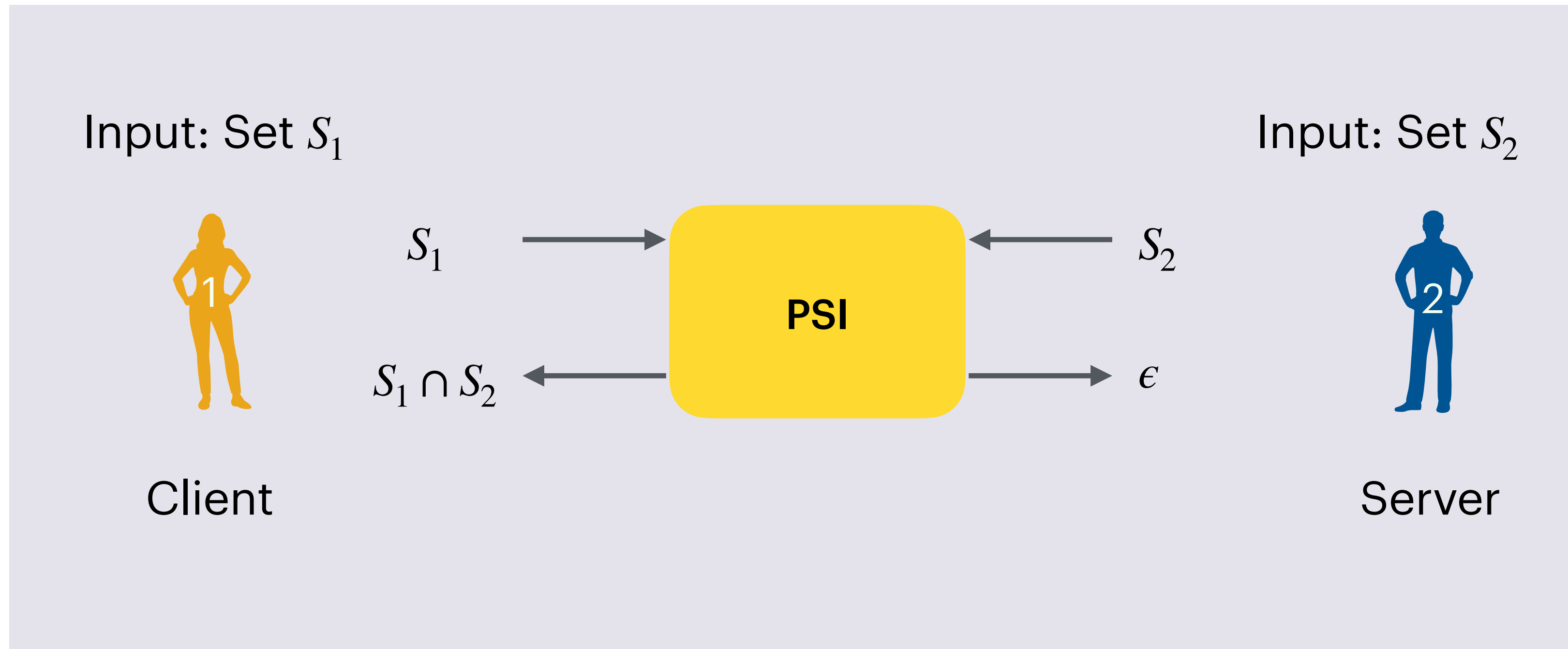
Mihir Bellare, University of California San Diego

Rishabh Ranjan, University of California San Diego

Doreen Riepel, CISPA



What is PSI?



Security

1. Party 1 (client) learns nothing about S_2 other than $S_1 \cap S_2$.
2. Party 2 (server) learns nothing about S_1 .

Why is PSI important?



Application	Client S_1		Server S_2	
Private Contact Discovery	App User	Personal Contacts	App Server	Contacts on the server
Credential Breach Monitoring	Service User	Personal Credentials	Service Provider	Breached Credentials
Ad Conversion Rates	Product Seller	Users who bought products	Advertiser	Users who clicked product advertisements
Personalized medicine	Individual	Personal Genomic Sequence	Hospital	Genetic fingerprint of drugs

How is security defined and measured?

Adversary = Dishonest party in the computation

How is security defined and measured?

Adversarial Setting

Semi Honest

Adversary assumed to follow the protocol but tries to learn about other party's input.

Malicious

Adversary can arbitrarily deviate from the protocol to learn about other party's input.

How is security defined and measured?

Adversarial Setting

Semi Honest

Adversary assumed to follow the protocol but tries to learn about other party's input.

Malicious

Adversary can arbitrarily deviate from the protocol to learn about other party's input.

How is security defined and measured?

Adversarial Setting

Semi Honest

Adversary assumed to follow the protocol but tries to learn about other party's input.

Malicious

Adversary can arbitrarily deviate from the protocol to learn about other party's input.

Definitions

Simulation

Traditionally used in MPC literature. Security requires **existence** of a **simulator**.

Input Indistinguishability

Simpler **indistinguishability-based** definition [MPRO6, BRRA24]. **No simulator!** Better suited for concrete security analyses.

How is security defined and measured?

Adversarial Setting

Semi Honest

Adversary assumed to follow the protocol but tries to learn about other party's input.

Malicious

Adversary can arbitrarily deviate from the protocol to learn about other party's input.

Definitions

Simulation

Traditionally used in MPC literature. Security requires **existence** of a **simulator**.



Input Indistinguishability

Simpler **indistinguishability-based** definition [MPRO6, BRRA24]. **No simulator!** Better suited for concrete security analyses.

Thm: [BRRA24] For **PSI**, the Input-Indistinguishability definition is **equivalent** to a simulation-based definition!

How is security defined and measured?

Adversarial Setting

Semi Honest

Adversary assumed to follow the protocol but tries to learn about other party's input.

Malicious

Adversary can arbitrarily deviate from the protocol to learn about other party's input.

Definitions

Simulation

Traditionally used in MPC literature. Security requires **existence** of a **simulator**.

Input Indistinguishability

Simpler indistinguishability-based definition [MPRO6, BRRA24]. **No simulator!** Better suited for concrete security analyses.

Prior Work in PSI

Constructions

1980s-90s: First

constructions given based
on DH [Mea86, HFH99]

2000s: PSI formally described
in [FIPR05] and OPRF based
construction given [HL08]

2010s: Oblivious Transfer
Extension based constructions
[PSZ14, PSSZ15, KKRT16, PRTY19,
CM20, PRTY20]

2020s: More efficient PSI
constructions based on VOLE
[GPRTY21, RS21, RR22, BPST23]

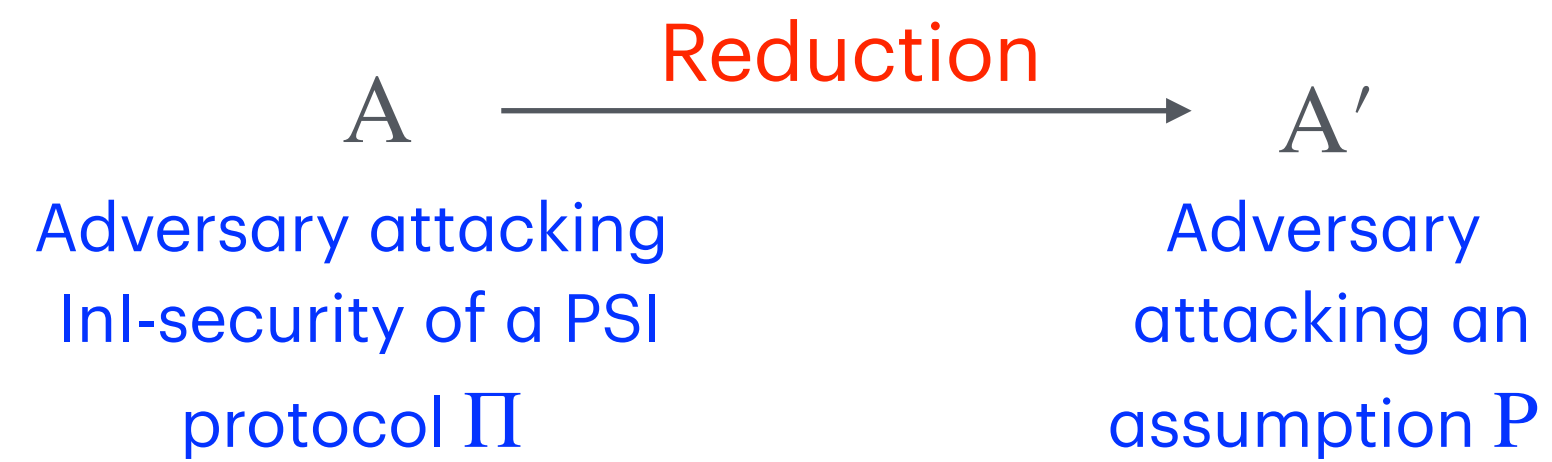
Theoretical

Practical and
being deployed

Limitations of Prior Work

Modern, Fast PSI protocols lack
concrete security analyses

Our contributions



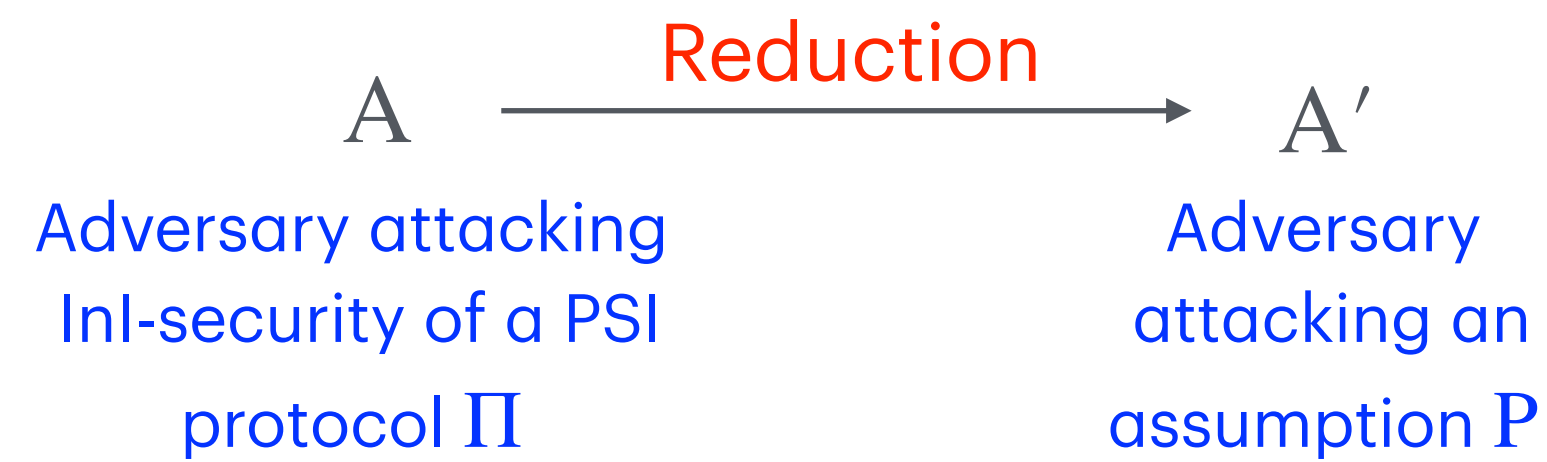
Asymptotic versus concrete security

Asymptotic Security Result

If A runs in
polynomial time
and has advantage that is
not negligible
then A' runs in
polynomial time
and has advantage that is
not negligible

Difficult to determine parameters
for Π for a desired level of security
in implementations.

Asymptotic versus concrete security



Asymptotic Security Result

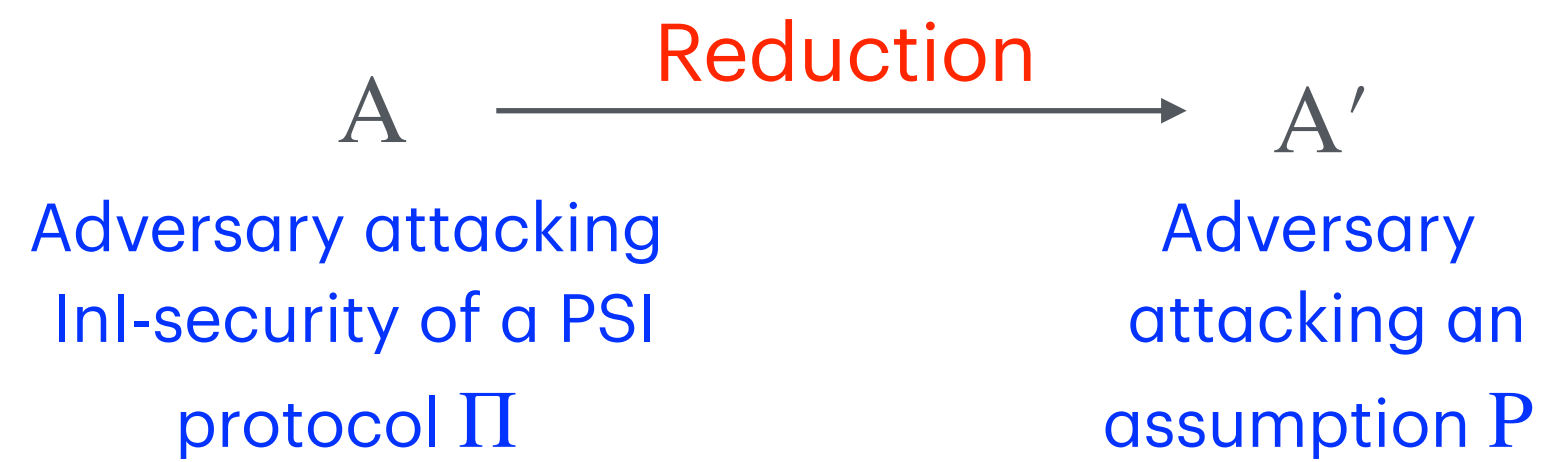
If A runs in
polynomial time
and has advantage that is
not negligible
then A' runs in
polynomial time
and has advantage that is
not negligible

Difficult to determine parameters
for Π for a desired level of security
in implementations.

Concrete Security Result

If A runs in time
 t
and has advantage that is
 $\epsilon = \text{Adv}_{\Pi}^T(A)$
then A' runs in time about
 t
and has advantage
 ϵ' such that $\epsilon \leq \mathbf{B}(\epsilon')$

Easy to determine parameters
for Π for a desired level of
security in implementations.



Asymptotic Security Result

If A runs in
polynomial time
and has advantage that is
not negligible
then A' runs in
polynomial time
and has advantage that is
not negligible

Difficult to determine parameters
for Π for a desired level of security
in implementations.

Concrete Security Result

If A runs in time
 t
and has advantage that is
 $\epsilon = \text{Adv}_{\Pi}^T(A)$
then A' runs in time about
 t
and has advantage
 ϵ' such that $\epsilon \leq \mathbf{B}(\epsilon')$

Easy to determine parameters
for Π for a desired level of
security in implementations.

Asymptotic versus concrete security

Concrete security the norm in symmetric
and asymmetric cryptography

PSI historically asymptotic

Concrete security for PSI begins with
[BRR24].

But they did not treat the modern, fast PSI
protocols.

We want to fill this gap.

Limitations of Prior Work

Modern, Fast PSI protocols lack
concrete security analyses

Our contributions

Provide **concrete security** analyses, with
reductions as tight as possible, for these protocols.

Limitations of Prior Work

Modern, Fast PSI protocols lack
concrete security analyses

Modern, Fast PSI protocols lack a
unifying, modular framework.

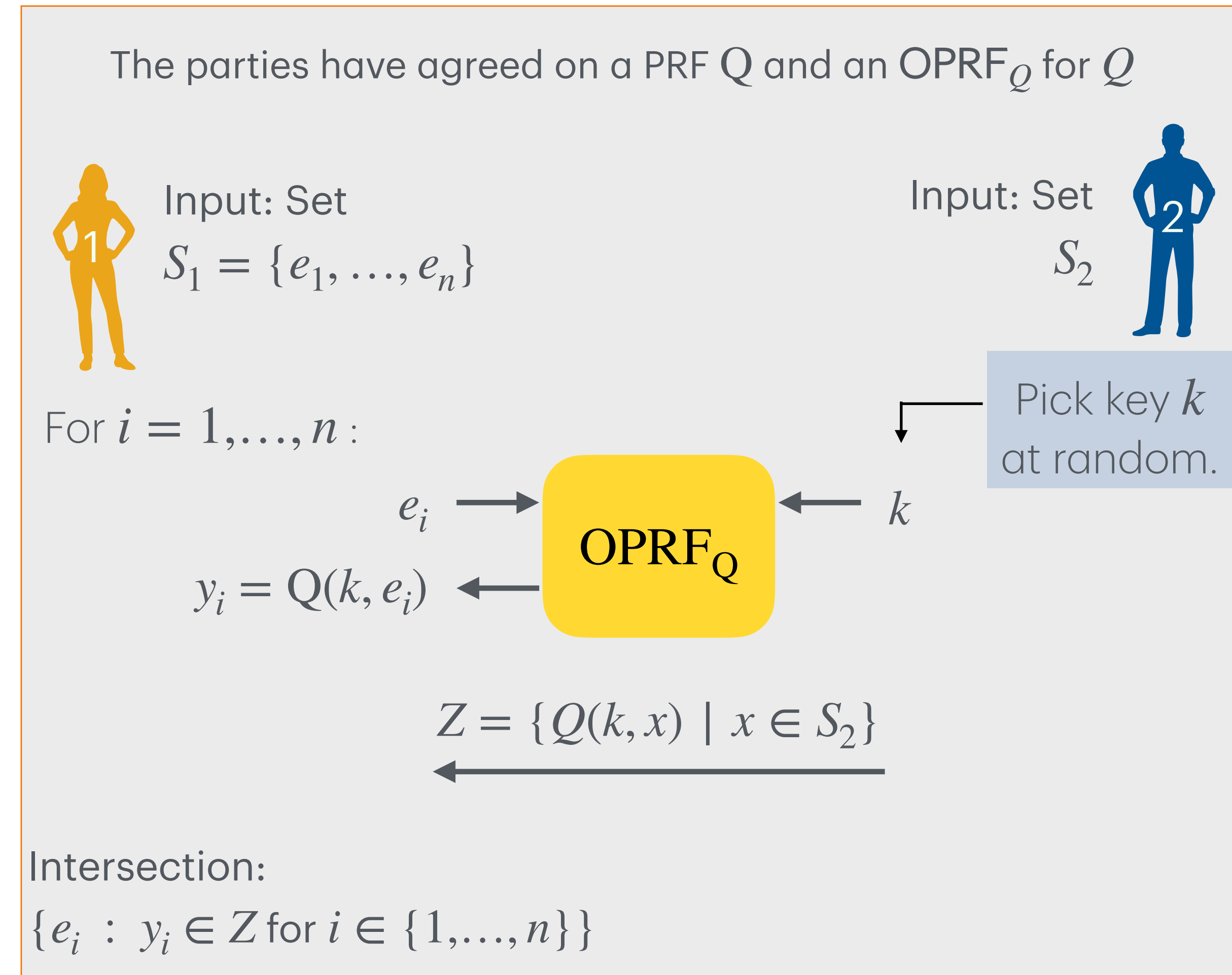
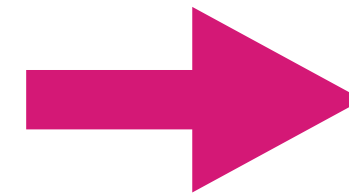
Our contributions

Provide **concrete security** analyses, with
reductions as tight as possible, for these protocols.

Modern, fast PSI protocols

PSZ14, PSSZ15, KKRT16, PRTY19, CM20,
PRTY20, GPRTY21, RS21, RR22, BPSY23

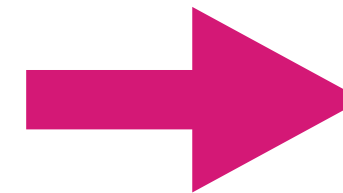
Start from the [OPRF-based PSI paradigm](#) [HL08]



Modern, fast PSI protocols

PSZ14, PSSZ15, KKRT16, PRTY19, CM20,
PRTY20, GPRTY21, RS21, RR22, BPSY23

Start from the **OPRF-based PSI paradigm** [HL08]

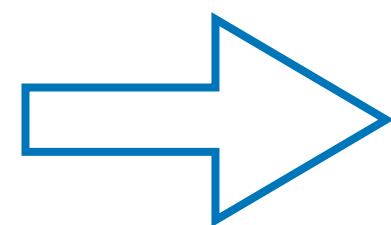


But, for efficiency, they **deviate from it**.

They might build some OPRF-like object.

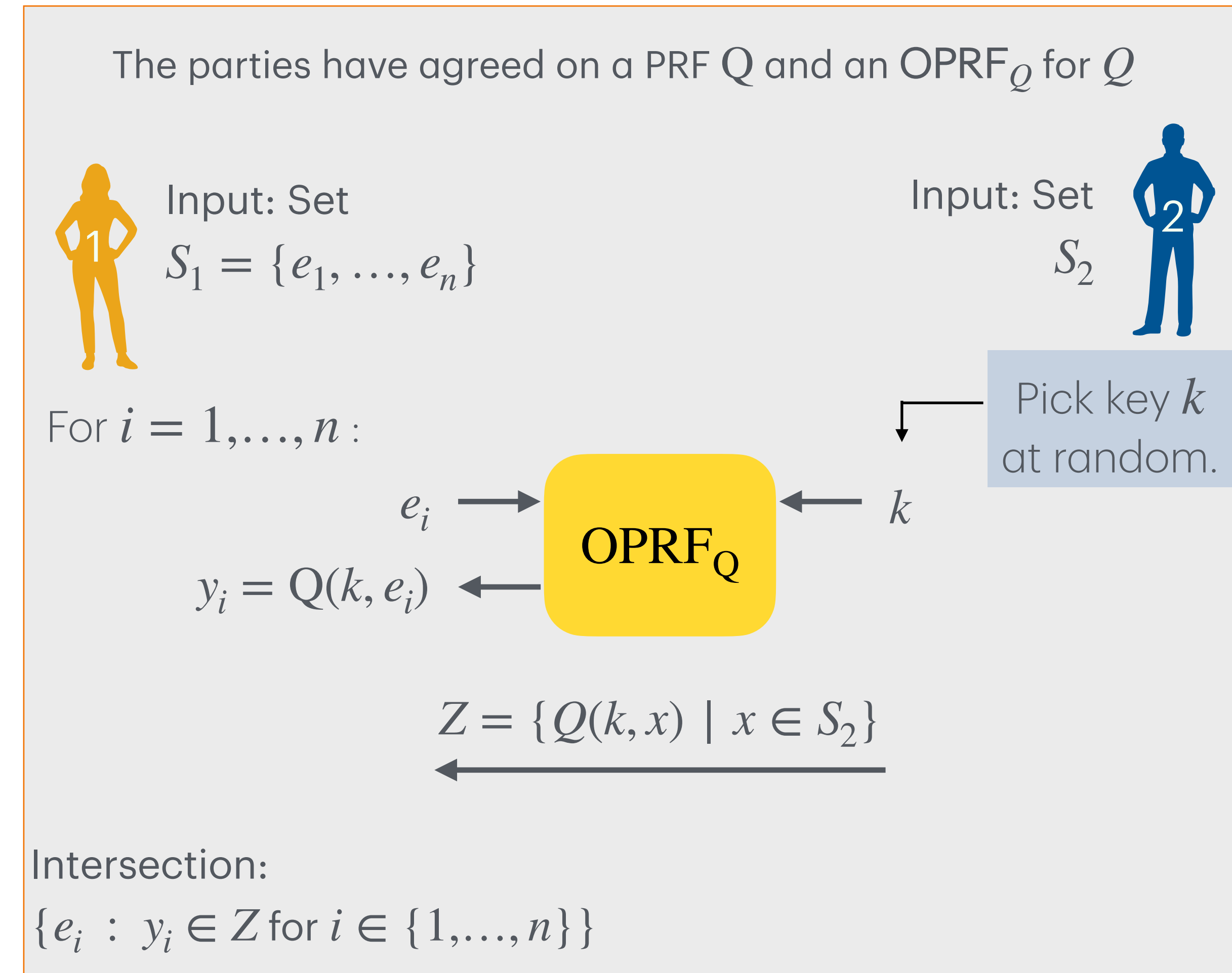
But security of the PSI is not reduced to that anyway.
Instead it is shown based lower level primitives (OT, VOLE,
PRF, ...)

Preferable: Find a **unifying, modular framework** to
obtain them all, generalizing the clean OPRF-
based PSI paradigm.



In particular avoids ad hoc concrete-
security analyses.

First Steps: BaRK (O)PRFs [KKRT16]



Limitations of Prior Work

Modern, Fast PSI protocols lack **concrete security** analyses

Modern, Fast PSI protocols lack a unifying, modular framework.

Our contributions

Provide **concrete security** analyses, with reductions as tight as possible, for these protocols.

We give the **JIT-OPRF-based PSI** framework:

1. We define a primitive we call a **JIT-OPRF**
2. We show how to **generically get PSI** from any JIT-OPRF
3. We **build JIT-OPRFs** from OT, VOLE and KVS
4. We recover the modern, fast PSI protocols and more
5. **Concrete-security proofs** throughout

Limitations of Prior Work

Modern, Fast PSI protocols lack **concrete security** analyses

Modern, Fast PSI protocols lack a unifying, modular framework.

Proof gaps

Optimized protocols not proven secure

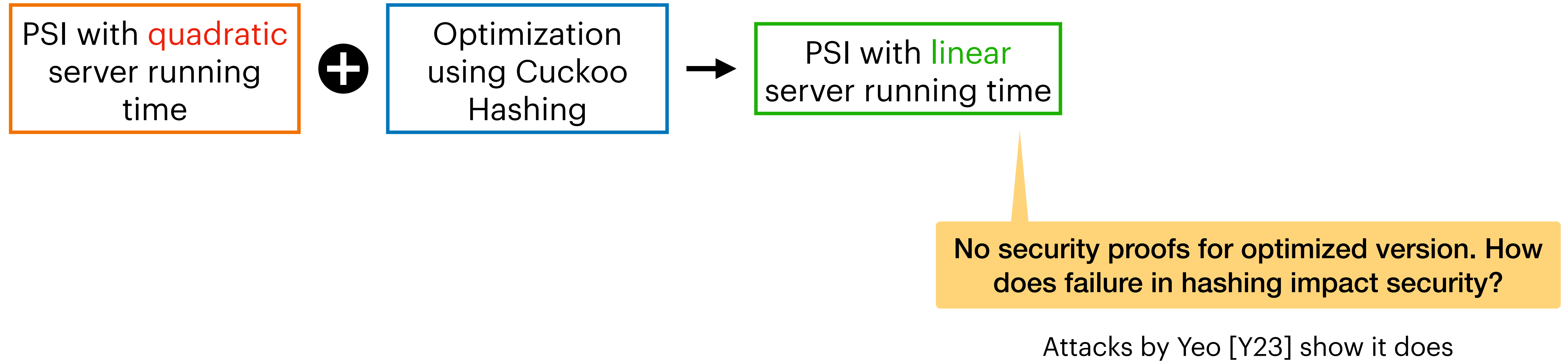
Our contributions

Provide **concrete security** analyses, with reductions as tight as possible, for these protocols.

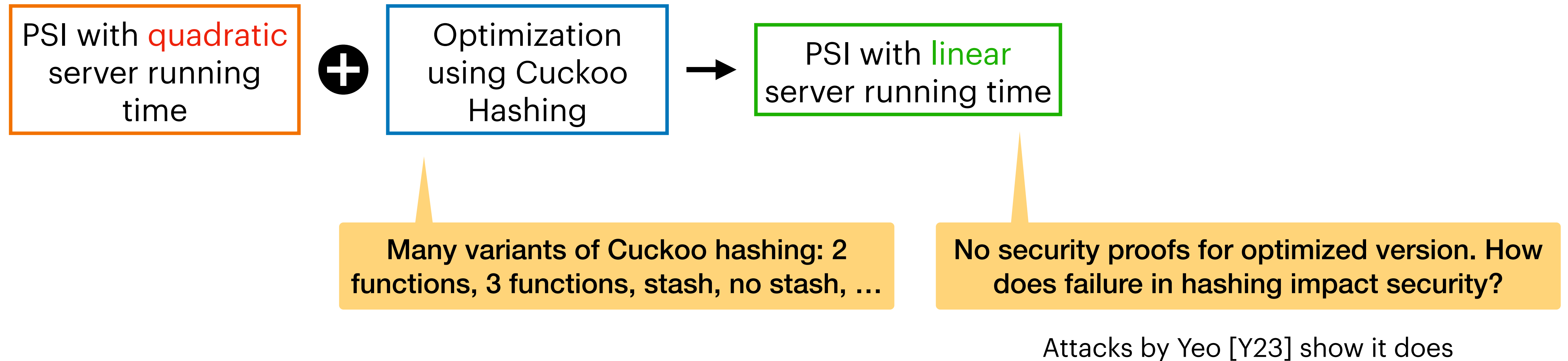
We give the **JIT-OPRF-based PSI** framework:

1. We define a primitive we call a **JIT-OPRF**
2. We show how to **generically get PSI** from any JIT-OPRF
3. We **build JIT-OPRFs** from OT, VOLE and KVS
4. We recover the modern, fast PSI protocols and more
5. **Concrete-security proofs** throughout

PSI construction process in prior work:



PSI construction process in prior work:



Limitations of Prior Work

Modern, Fast PSI protocols lack **concrete security** analyses

Modern, Fast PSI protocols lack a unifying, modular framework.

Proof gaps

Optimized protocols not proven secure

Our contributions

Provide **concrete security** analyses, with reductions as tight as possible, for these protocols.

We give the **JIT-OPRF-based PSI** framework:

1. We define a primitive we call a **JIT-OPRF**
2. We show how to **generically get PSI** from any JIT-OPRF
3. We **build JIT-OPRFs** from OT, VOLE and KVS
4. We recover the modern, fast PSI protocols and more
5. **Concrete-security proofs** throughout

We fill the gaps:

1. We define a primitive we call a **bin-allocation** scheme
2. We derive the optimized protocols from **any such** scheme
3. We prove security of (the basic and the) optimized protocols
4. We show how Cuckoo hashing yields a bin allocation scheme

Our Contributions in brief

Definitions: JIT-OPRFs and Bin Allocation

- Game-based security definitions for JIT-PRF and JIT-OPRFs.
- JIT-PRFs define a parameter *capacity* which can be 1 or ∞ .

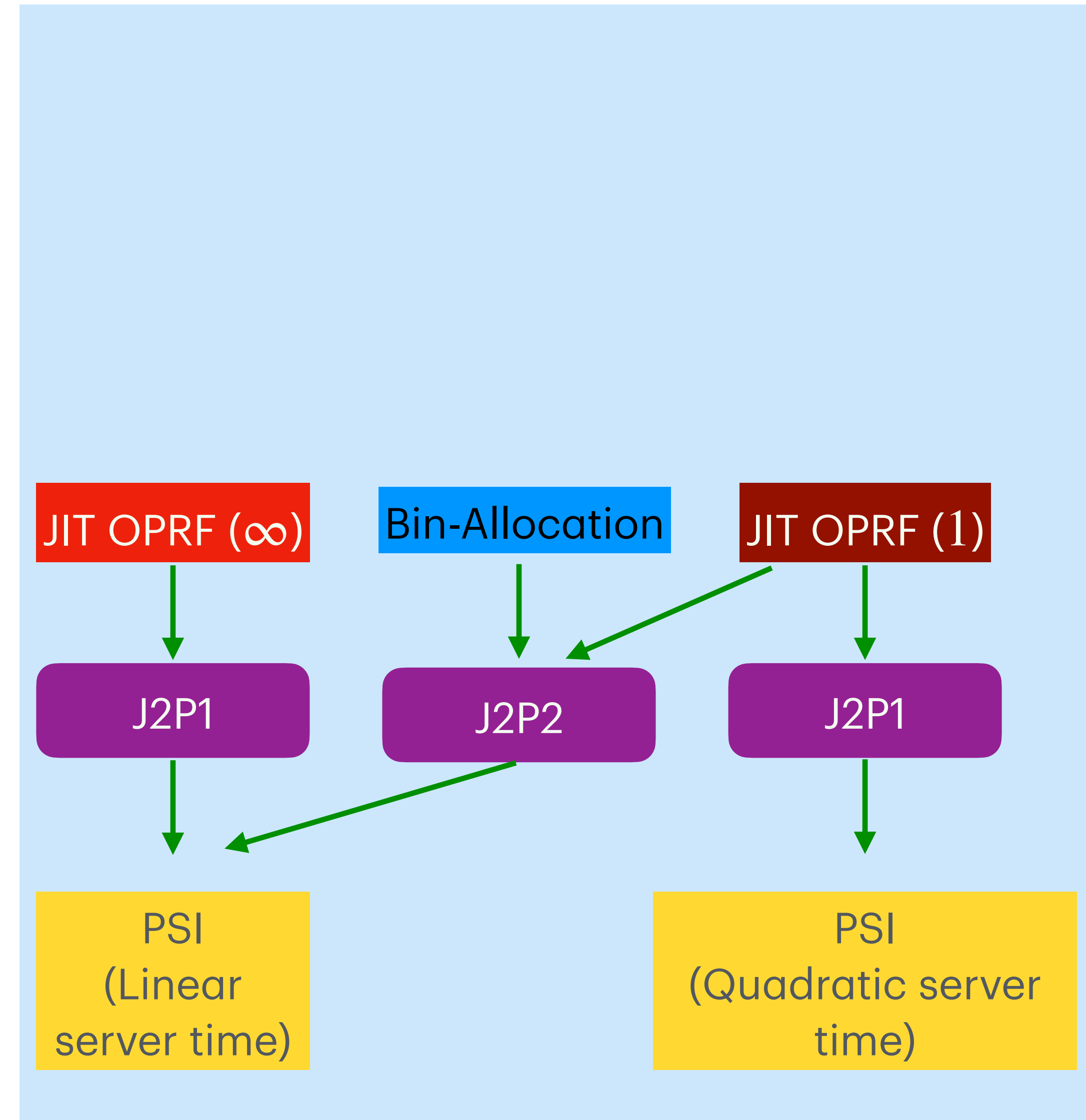
Our Contributions in brief

☑ Definitions: JIT-OPRFs and Bin Allocation

- ▶ Game-based security definitions for JIT-PRF and JIT-OPRFs.
- ▶ JIT-PRFs define a parameter *capacity* which can be 1 or ∞ .

☑ PSI generically from JIT-OPRFs

- ▶ **J2P1**: PSI directly from JIT-OPRFs. With capacity 1 JIT-PRF we get PSI with quadratic server running time and with capacity ∞ JIT-PRF we get PSI with linear server running time.
- ▶ **J2P2**: PSI from bin-allocation and JIT-OPRFs based on JIT-PRFs with capacity 1. Builds PSI with linear server running time.



Our Contributions in brief

☑ Definitions: JIT-OPRFs and Bin Allocation

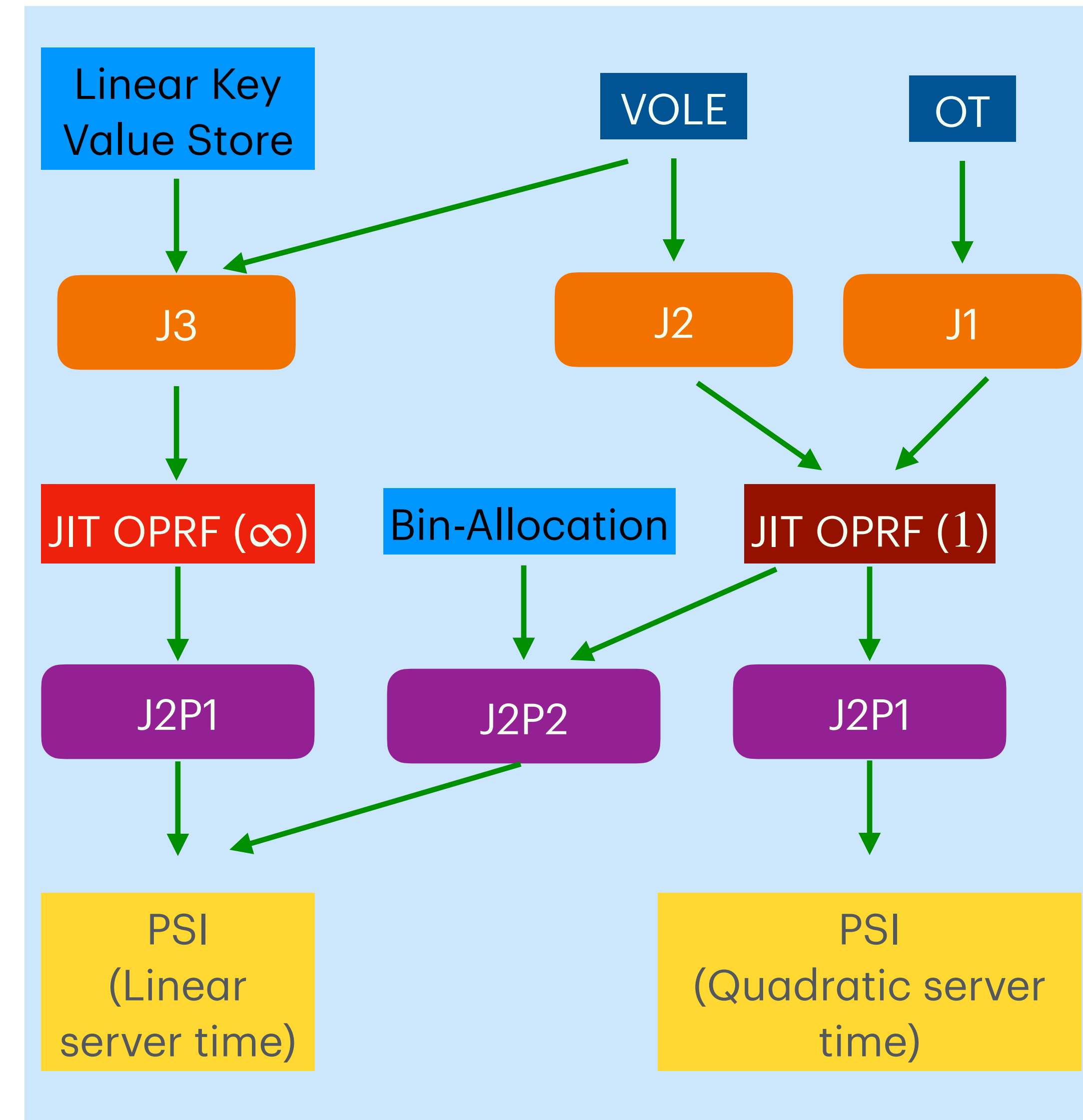
- ▶ Game-based security definitions for JIT-PRF and JIT-OPRFs.
- ▶ JIT-PRFs define a parameter *capacity* which can be 1 or ∞ .

☑ PSI generically from JIT-OPRFs

- ▶ **J2P1**: PSI directly from JIT-OPRFs. With capacity 1 JIT-PRF we get PSI with quadratic server running time and with capacity ∞ JIT-PRF we get PSI with linear server running time.
- ▶ **J2P2**: PSI from bin-allocation and JIT-OPRFs based on JIT-PRFs with capacity 1. Builds PSI with linear server running time.

☑ JIT-OPRF constructions based on fast PSI

- ▶ **J1**: built from Oblivious Transfer. Based on PSI protocol of [KKRT16].
- ▶ **J2**: built only from Vector Oblivious Linear Evaluation (VOLE).
- ▶ **J3**: built from Linear Key Value Stores and VOLE. Based on PSI protocols of [GPRTY21, RS21, RR22, BPSY23].



Quick Closer Look

Just-in-Time PRF

A JIT family of functions Q defines:

Evaluation



 Key Generation



 Capacity

1 or ∞ ;
determines
number of
output keys.

$$\ell = \begin{cases} n & \text{If capacity is 1} \\ 1 & \text{Otherwise} \end{cases}$$

★ We define **PRF security of JIT-PRFs** similar to the usual multi-user PRF game.

JIT-PRFs capture as special cases BaRK-PRFs [KKRT16] and multi-input PRFs [RS21].

Just-in-Time PRF

A JIT family of functions Q defines:

Evaluation



New Key Generation



New Capacity

1 or ∞ ;
 determines
 number of
 output keys.

$$\ell = \begin{cases} n & \text{If capacity is 1} \\ 1 & \text{Otherwise} \end{cases}$$

Just-in-Time OPRF

Input: $X = (x_1, \dots, x_n)$



Client

Input: ε



Server

$x_1, \dots, x_n$ →

JIT-OPRF
 based on Q

← ε

$y_1, \dots, y_n$ ←

→ $k_1, \dots, k_\ell$

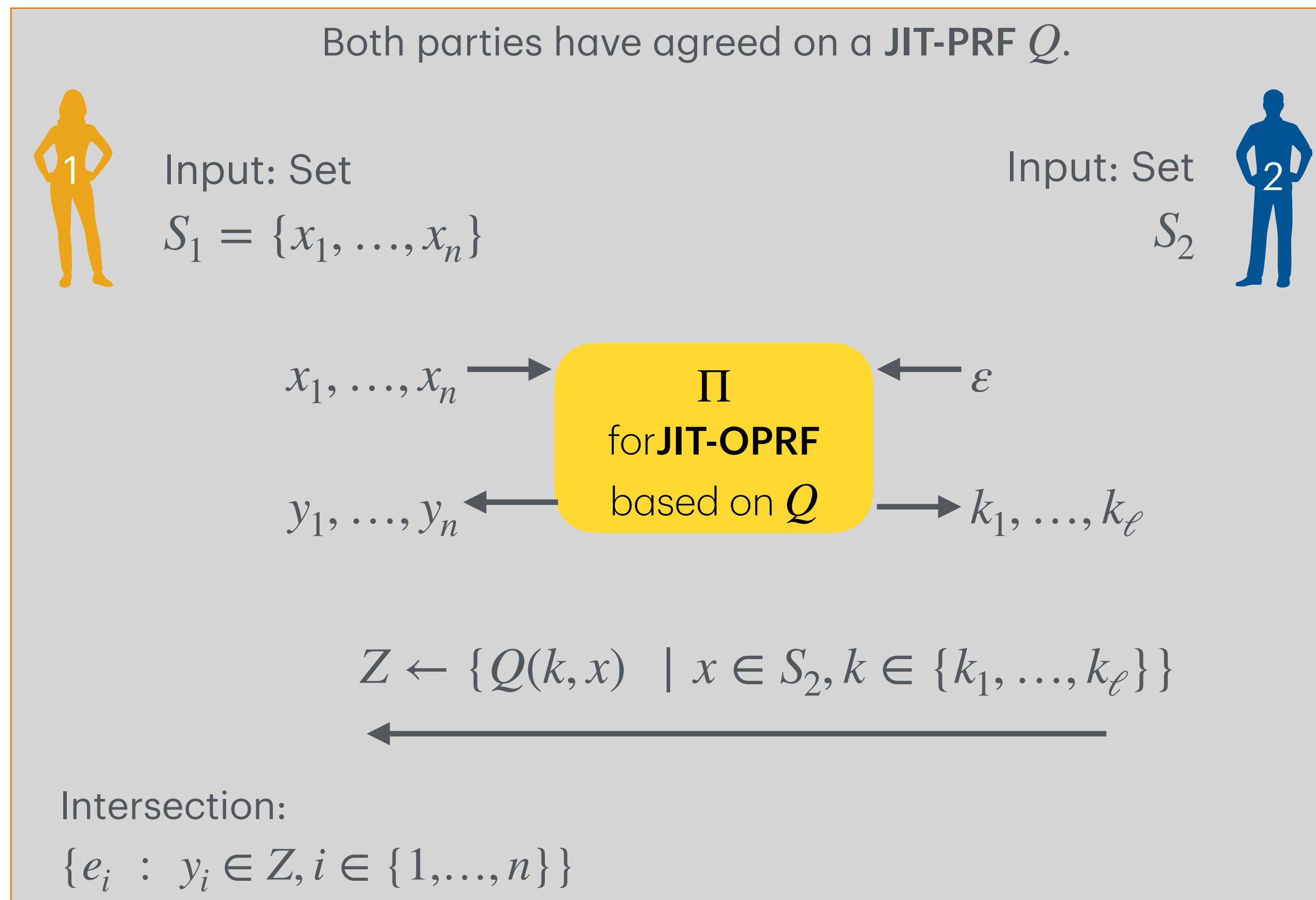
$$// y_i = \begin{cases} Q(k_i, e_i) & \text{If capacity is 1} \\ Q(k_1, e_i) & \text{Otherwise} \end{cases}$$

We define **security of JIT-OPRFs** via games :

- ➡ Party 1 security: privacy of inputs $(x_1, \dots, x_n)$
- ➡ Party 2 security: pseudo-randomness of $Q(k_j, \cdot)$

JIT-OPRFs capture as special cases
 BaRK-OPRFs [KKRT16] and multi-input
 OPRFs [RS21].

PSI from JIT-OPRFs: J2P1 Transform



▶ Theorem (Informal)

Inl security of PSI protocol output by $J2P1[Q, \Pi]$ is implied by the JIT-OPRF security of the underlying JIT-OPRF protocol Π .

✓ This is for both client and server.

✓ Security is tight.

◆ With Unit Capacity Q

Server does quadratic computation

◆ With Unbounded Capacity Q

Server does linear computation

PSI from JIT-OPRFs: J2P2 Transform

Both parties have agreed on a JIT-PRF Q and bin allocation scheme BS.



Input: Set

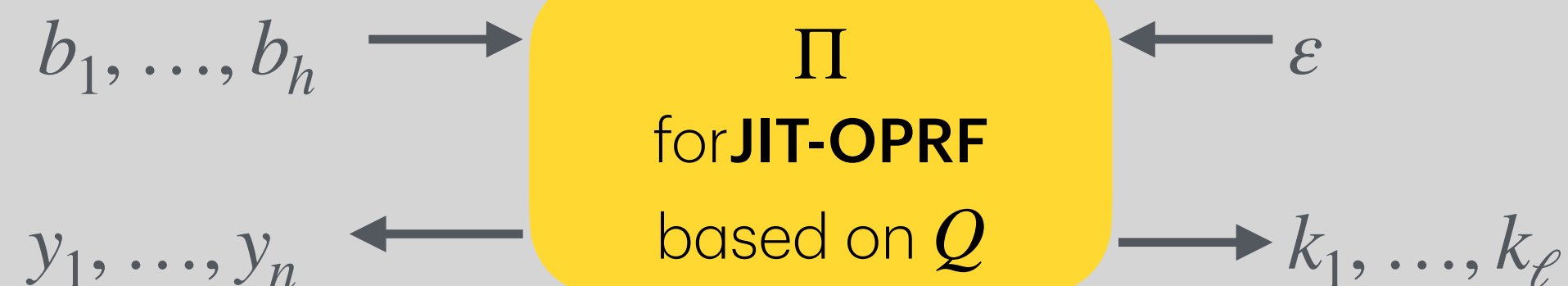
$$S_1 = \{x_1, \dots, x_n\}$$

Allocate S_1 using BS to construct
JIT-OPRF input $b_1, \dots, b_h$



Input: Set

$$S_2$$



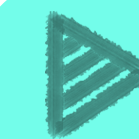
$$Z \leftarrow \{Q(k, x) \mid x \in S_2, k \in P(x)\}$$

Intersection:

$$\{e_i : y_i \in Z, i \in \{1, \dots, n\}\}$$

$P(x)$ set of possible keys
computed using BS

$|P(x)|$ is generally a constant.



Theorem (Informal)

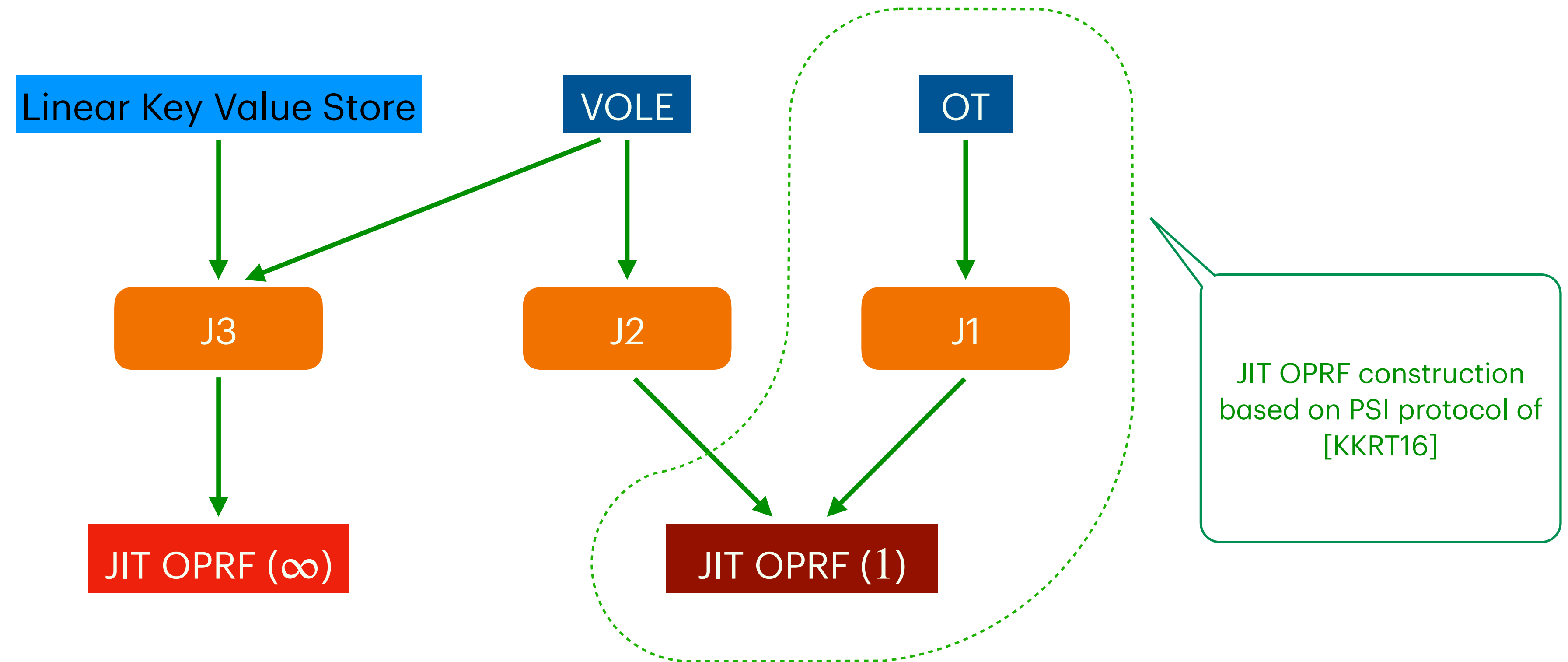
Informal security of PSI protocol
output by $J2P2[Q, \Pi, BS]$
is implied by the JIT-OPRF
security of the underlying
JIT-OPRF protocol Π and
the correctness of BS.

- ✓ Correctness of BS is needed for client security.
- ✓ Security is tight.

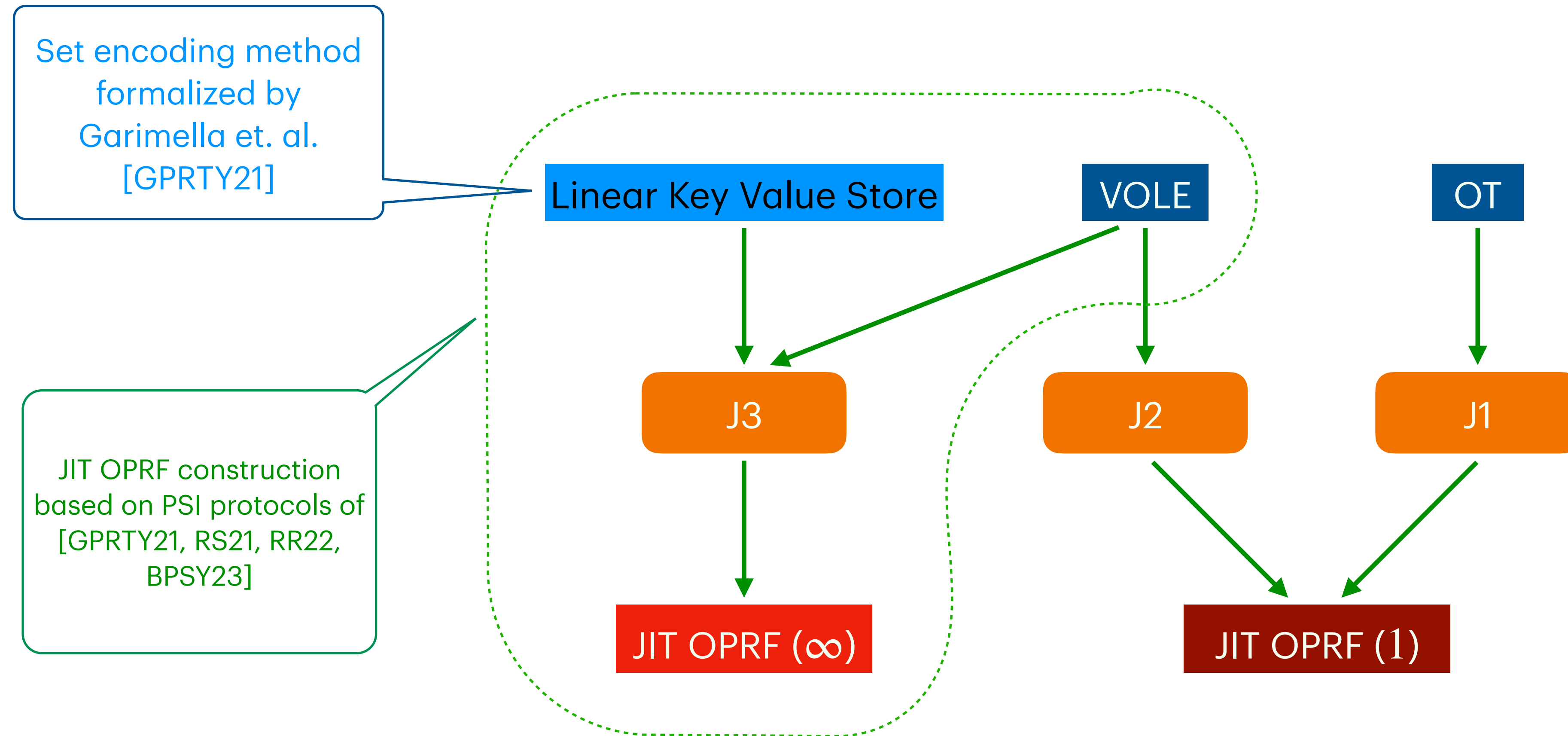
Our result explicitly show client security loss due to correctness error in BS. This has not been discussed in prior works.

Server does linear computation

JIT-OPRF Constructions



JIT-OPRF Constructions



Results for JIT-OPRF Constructions - Client Security

Concrete Security

Given an adversary A_{jc} playing the JIT-OPRF-1 game,

we construct adversary A_P playing the InI game with honest party 1 for protocol of underlying functionality P

such that advantage of A_{jc} is bounded by advantage ϵ_P of A_P as

JIT-OPRF Protocol	Bound
From OT	ϵ_{OT}
From VOLE	ϵ_{VOLE}
From LKVS and VOLE	ϵ_{VOLE}

- ★ Tight security for all.
- ★ No property for LKVS is needed.

Results for JIT-OPRF Constructions - Server Security

Concrete Security

Given an adversary A_{js} playing the JIT-OPRF-2 game,

we construct adversary A_P playing the InI game with honest party 2 for protocol of underlying functionality P

such that advantage of A_{js} is bounded by advantage ϵ_P of A_P as

JIT-OPRF Protocol	Bound	
	Computational Term	Statistical Term
From OT	ϵ_{OT}	δ_1
From VOLE	ϵ_{VOLE}	δ_2
From LKVS and VOLE	$\phi_{KVS} + \epsilon_{VOLE}$	δ_3

★ Tight computational bounds.

Server security needs a new property for KVS, that we define and call **Encoding Unforgeability**

Comparison of statistical terms in Server security results

Resource usage of A_{js} :

- Q_T transcript oracle queries
- Q_C challenge oracle queries
- Q_H random oracle queries

- L sum of input sizes across all transcript oracle queries
- p size of range of JIT-PRF evaluation function

For computation of p we assume:

- $Q_T \approx 2^{10}, L \approx 2^{10} \cdot Q_T \approx 2^{20}$
- $Q_C \approx 2^{20}$
- $Q_H \approx 2^{60}$

	δ_1 (J1)	δ_2 (J2)	δ_3 (J3)
Equations	$\frac{L \cdot (Q_C + Q_H)}{p^{1/5}} + \frac{Q_H^2}{2p^{1/4}} + \frac{L^2}{2p}$	$\frac{Q_H^2 + 4L \cdot (Q_C + Q_H) + 2Q_T + L^2}{2p}$	$\frac{Q_T \cdot (1 + 2Q_C + 3Q_H)}{p}$
Log of p to make the statistical terms $\leq 2^{32}$	608	151	103

Comparison of statistical terms in Server security results

Resource usage of A_{js} :

- Q_T transcript oracle queries
- Q_C challenge oracle queries
- Q_H random oracle queries

- L sum of input sizes across all transcript oracle queries
- p size of range of JIT-PRF evaluation function

For computation of p we assume:

- $Q_T \approx 2^{10}, L \approx 2^{10} \cdot Q_T \approx 2^{20}$
- $Q_C \approx 2^{20}$
- $Q_H \approx 2^{60}$

	δ_1 (J1)	δ_2 (J2)	δ_3 (J3)
Equations	$\frac{L \cdot (Q_C + Q_H)}{p^{1/5}} + \frac{Q_H^2}{2p^{1/4}} + \frac{L^2}{2p}$	$\frac{Q_H^2 + 4L \cdot (Q_C + Q_H) + 2Q_T + L^2}{2p}$	$\frac{Q_T \cdot (1 + 2Q_C + 3Q_H)}{p}$
Log of p to make the statistical terms $\leq 2^{32}$	608	151	104

★ Our concrete security results explicitly show LKVS+VOLE JIT-OPRF (and so PSI) allows working with much smaller domains for the same security level.

Summary

☑ Definitions: JIT-OPRFs and Bin Allocation

- ▶ Game-based security definitions for JIT-PRF and JIT-OPRFs.
- ▶ JIT-PRFs define a parameter *capacity* which can be 1 or ∞ .

☑ PSI generically from JIT-OPRFs

- ▶ **J2P1**: PSI directly from JIT-OPRFs. With *capacity 1* JIT-PRF we get PSI with quadratic server running time and with *capacity ∞* JIT-PRF we get PSI with linear server running time.
- ▶ **J2P2**: PSI from bin-allocation and JIT-OPRFs based on JIT-PRFs with *capacity 1*. Builds PSI with linear server running time.

☑ JIT-OPRF constructions based on fast PSI

- ▶ **J1**: built from Oblivious Transfer. Based on PSI protocol of [KKRT16].
- ▶ **J2**: built only from Vector Oblivious Linear Evaluation (VOLE).
- ▶ **J3**: built from Linear Key Value Stores and VOLE. Based on PSI protocols of [GPRTY21, RS21, RR22, BPSY23].

